

爬虫系统项目介绍

一、整体架构

1.1 技术栈

层级	技术	说明
后端框架	Flask 3.0.0	Python Web框架
数据库	SQLite	轻量级关系型数据库
跨域支持	Flask-CORS	处理跨域请求
密码加密	bcrypt	用户密码安全存储
定时任务	APScheduler 3.10.4	爬虫定时调度
爬虫引擎	requests + BeautifulSoup4	网页抓取与解析
可视化	matplotlib + wordcloud	图表生成
前端	原生HTML/CSS/JS	无框架依赖

1.2 系统架构层次

系统采用经典的分层架构，从上到下依次为：

- 前端展示层：**HTML页面 + CSS样式 + JavaScript交互
- 路由控制层：**Flask路由处理HTTP请求
- 业务逻辑层：**CrawlerService爬虫服务 + AIModelCaller AI分析 + DataVisualizer可视化
- 数据访问层：**Model类封装数据库操作
- 数据存储层：**SQLite数据库持久化存储

1.3 项目目录结构

```
d:\python\Crawl\  
├── sign.html           # 登录注册页面  
├── index.html          # 主功能页面  
├── show.html           # 可视化展示页面  
├── check.html          # 结果查看页面  
├── viz_list.html       # 可视化列表页面  
├── backend\  
│   ├── app.py         # Flask主应用  
│   ├── call_ai.py     # AI模型调用模块  
│   ├── show.py        # 数据可视化模块  
│   └── models\  
│       ├── database.py # 数据库连接
```

```
├── user.py          # 用户模型
├── config_file.py   # 配置文件模型
├── crawler_task.py  # 爬虫任务模型
├── crawler_result.py # 结果模型
├── ai_config.py     # AI配置模型
├── visualization.py # 可视化模型
├── services\       # 业务逻辑层
│   └── crawler_service.py # 爬虫服务
├── static\         # 静态资源
│   ├── css\        # 样式文件
│   └── js\          # JavaScript脚本
├── database\       # 数据库文件
│   └── crawl.db     # SQLite数据库
```

1.4 数据库表结构

表名	用途	核心字段
users	用户管理	id, username, password, create_time
config_files	配置文件存储	id, user_id, name, type, content, create_time
crawler_tasks	爬虫任务记录	id, user_id, config_id, time_config_id, status, start_time, end_time
crawler_results	爬虫结果存储	id, task_id, content, is_saved, create_time
ai_configs	AI模型配置	id, user_id, name, config_type, base_url, api_key, model_name
visualizations	可视化数据	id, result_id, chart_type, data, create_time

二、各个模块分别讲解

2.1 用户认证模块

文件位置：backend/models/user.py + static/js/auth.js

功能说明：

- 用户注册：使用bcrypt对密码进行加密存储
- 用户登录：验证用户名密码，创建Session会话
- 登录状态检查：前端检查Session有效性，未登录自动跳转
- 退出登录：清除Session，返回登录页

核心方法：

方法	功能	说明
User.register(username, password)	用户注册	检查用户名唯一性，密码bcrypt加密
User.login(username, password)	用户登录	验证密码，返回用户对象
User.verify_password()	密码验证	bcrypt.checkpw比对

路由接口:

- POST /api/register - 用户注册
 - POST /api/login - 用户登录
 - POST /api/logout - 退出登录
 - GET /api/user/info - 获取用户信息
-

2.2 配置管理模块

文件位置: backend/models/config_file.py

功能说明:

- 支持两种配置类型: 爬虫配置(crawler)和时间配置(time)
- 支持两种方式上传: 文本框直接编辑JSON、文件上传
- 支持查看、编辑、删除已上传的配置文件

爬虫配置文件格式:

```
{
  "type": "crawler",
  "name": "爬虫名称",
  "urls": ["https://example.com/page1"],
  "selectors": {
    "title": "h1, h2, .title",
    "content": "p, .content"
  },
  "headers": {
    "User-Agent": "Mozilla/5.0..."
  },
  "delay": 2,
  "max_pages": 10,
  "timeout": 30,
  "retry_times": 3
}
```

时间配置文件格式:

```
{
  "type": "time",
  "name": "定时配置名称",
  "cron": "0 9 * * *",
  "description": "每天早上9点执行"
}
```

路由接口:

- POST /api/config/upload - 上传配置

- GET /api/config/list - 获取配置列表
- GET /api/config/ - 获取单个配置详情
- PUT /api/config/ - 更新配置
- DELETE /api/config/ - 删除配置

2.3 爬虫任务模块

文件位置: backend/services/crawler_service.py + backend/models/crawler_task.py

功能说明:

2.3.1 爬虫服务类 CrawlerService

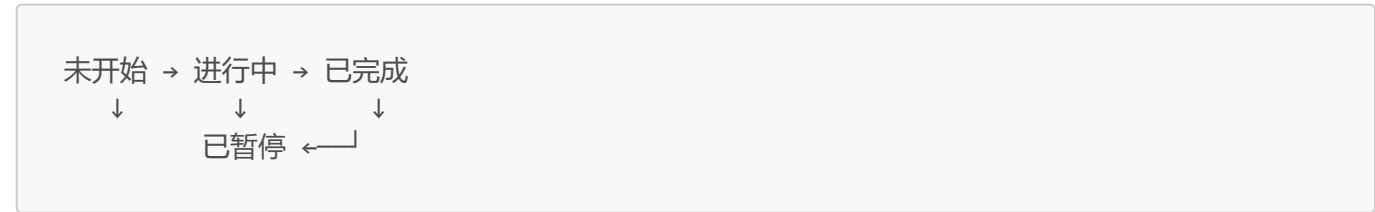
采用单例模式设计，确保全局只有一个爬虫调度器实例。

方法	功能	说明
start_task(task_id, config)	立即启动爬虫	创建线程执行爬虫任务
schedule_task(task_id, config, time_config)	定时启动爬虫	使用APScheduler设置Cron定时
pause_task(task_id)	暂停任务	设置暂停标志位
resume_task(task_id, config)	恢复任务	清除暂停标志，重新启动
stop_task(task_id)	停止任务	移除定时任务，更新状态

2.3.2 爬虫执行流程

1. 解析配置：提取urls、selectors、headers等参数
2. 遍历URL：逐个请求目标网页
3. 重试机制：请求失败时自动重试（默认3次）
4. 解析页面：使用BeautifulSoup按选择器提取数据
5. 支持嵌套选择器：可配置多级数据提取规则
6. 延时控制：防止请求过快被封禁
7. 保存结果：爬取完成后存入crawler_results表

2.3.3 任务状态流转



任务状态: 未开始、进行中、已完成、已暂停

路由接口:

- POST /api/task/create - 创建任务
- POST /api/task//start - 启动任务

- POST /api/task//pause - 暂停任务
- POST /api/task//resume - 恢复任务
- POST /api/task//stop - 停止任务
- GET /api/task/list - 获取任务列表

2.4 AI分析模块

文件位置： backend/call_ai.py

功能说明：

- 支持两种AI调用方式：本地模型、API接口
- 对爬虫数据进行智能分析和清理
- 支持多种分析任务类型

AIModelCaller类核心方法：

方法	功能
clean_and_analyze_data(data, task_type)	清理和分析数据
_build_prompt(data, task_type)	构建AI提示词
_call_api_model(prompt)	调用API模型
_call_local_model(prompt)	调用本地模型
_parse_response(response, original_data)	解析AI返回结果

支持的分析任务类型：

- general：通用分析
- sentiment：情感分析
- keyword：关键词提取
- summary：摘要生成

配置参数：

- config_type： 'local' 或 'api'
- base_url： API基础地址
- api_key： API密钥
- model_name： 模型名称

2.5 可视化模块

文件位置： backend/show.py

功能说明：

- 使用matplotlib生成各类统计图表
- 支持词云生成（需安装wordcloud库）

- 图表以Base64编码返回，前端直接展示

DataVisualizer类图表类型：

方法	图表类型	用途
generate_wordcloud()	词云图	展示关键词分布
generate_sentiment_pie()	饼图	情感分析占比
generate_category_bar()	柱状图	分类数据统计
generate_keyword_bar()	条形图	热门关键词TOP15
generate_stats_chart()	组合图	综合统计信息
generate_timeline()	折线图	时间趋势分析

中文支持：

- 自动检测系统字体（Windows: simhei/msyh, Linux: wqy-zenhei, macOS: PingFang）
- 解决中文乱码和负号显示问题

路由接口：

- GET /api/result//visualize - 生成可视化图表
- GET /api/result/ - 获取结果详情

2.6 前端页面模块

2.6.1 sign.html - 登录注册页

功能：

- 用户注册表单（用户名、密码、确认密码）
- 用户登录表单（用户名、密码）
- 表单验证和错误提示
- 登录成功后跳转到index.html

2.6.2 index.html - 主功能页

功能区域：

区域	功能描述
上传配置区	支持爬虫配置和时间配置的文本编辑和文件上传
配置列表区	Scroll展示已上传的配置，支持查看、编辑、删除
示例配置区	显示和下载示例配置文件
爬虫执行区	选择配置启动爬虫，支持定时执行
任务状态区	显示任务状态（未开始/进行中/已完成/已暂停），已完成任务支持可视化和保存

2.6.3 show.html - 可视化展示页

布局：

- 左侧：结果列表（已保存和未保存的结果）
- 中间：可视化图表展示区域
- 支持点击切换展示不同的结果数据

JS文件分工：

文件	功能
auth.js	登录状态检查和用户认证
main.js	index.html主页面交互逻辑
show.js	show.html可视化页面逻辑
check.js	check.html结果查看逻辑

三、特色亮点分析

3.1 架构设计亮点

3.1.1 单例模式确保资源唯一性

CrawlerService采用单例模式，确保系统中只有一个爬虫调度器实例，避免资源冲突和重复调度。

```
class CrawlerService:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
```

3.1.2 分层清晰的MVC架构

- Model：数据模型层，封装数据库操作
- View：HTML页面，负责展示
- Controller：Flask路由，处理请求转发

3.1.3 完善的异常处理机制

爬虫执行过程中包含多层级异常捕获：

- 请求层：超时、连接错误处理
- 解析层：选择器异常处理

- 数据层：保存失败处理

3.2 爬虫功能亮点

3.2.1 灵活的配置系统

- 支持嵌套选择器：可提取复杂结构化数据
- 可配置请求头：模拟真实浏览器访问
- 智能延时策略：防止被封禁
- 自动重试机制：失败自动重试（默认3次）

3.2.2 强大的定时调度

- 基于APScheduler实现
- 支持Cron表达式：可配置任意复杂的定时规则
- 任务持久化：重启后定时任务不丢失

3.2.3 多线程并发执行

每个爬虫任务在独立线程中执行：

- 避免阻塞主线程
- 支持多任务并行
- 任务间互不干扰

3.3 AI集成亮点

3.3.1 双模式AI支持

- 本地模型：适合内网环境，数据不出域
- API模式：支持OpenAI兼容接口，灵活选择模型

3.3.2 智能数据清洗

AI自动完成：

- 数据去重和格式化
- 情感倾向分析
- 关键词自动提取
- 内容摘要生成

3.4 可视化亮点

3.4.1 丰富的图表类型

支持6种图表类型，覆盖常见数据展示需求：

- 词云：直观展示关键词权重
- 饼图：占比分析
- 柱状图/条形图：排名对比

- 折线图：趋势分析

3.4.2 完善的中文支持

- 自动检测系统字体
- 解决matplotlib中文乱码
- 支持负号正确显示

3.4.3 零依赖展示

图表生成Base64编码图片，前端无需额外图表库。

3.5 用户体验亮点

3.5.1 多种配置输入方式

- 文本框直接编辑JSON
- 文件上传JSON文件
- 提供示例配置参考

3.5.2 实时任务状态反馈

- 任务状态实时更新
- 详细的调试日志输出
- 错误信息友好提示

3.5.3 数据管理灵活

- 爬虫结果可选择保存或临时查看
- 历史结果随时可视化回顾
- 支持结果对比分析

3.6 安全设计亮点

3.6.1 密码安全

- 使用bcrypt加密存储
- 自动加盐防止彩虹表攻击
- 密码强度验证

3.6.2 会话管理

- 基于Session的登录状态
- 24小时会话有效期
- 退出即销毁会话

3.6.3 数据隔离

- 用户数据完全隔离

- 只能访问自己的配置和结果
- 数据库外键约束保证数据一致性

四、快速开始

4.1 安装依赖

```
pip install -r requirements.txt
```

4.2 启动服务

```
cd backend  
python app.py
```

4.3 访问系统

浏览器打开：<http://127.0.0.1:5000>

默认进入登录注册页面。

五、总结

本爬虫系统采用前后端分离架构，后端使用Flask框架，前端使用原生Web技术。系统具备完整的用户管理、配置管理、爬虫执行、AI分析和数据可视化功能。架构清晰、功能完善、易于扩展，适合作为爬虫项目的基础框架进行二次开发。